

# Bright Coffee Shop Sales

## SQL Exercise: Wildcards & Window Functions

Databricks Hands-On Worksheet

Name: MUZI TSOOBOLO

Due Date: Tuesday, 19 May 2026

Score: \_\_\_\_ / 20

About this dataset

You are working with sales data from Bright Coffee Shop, a chain with three locations:

Lower Manhattan | Hell's Kitchen | Astoria

The table is called: coffee\_sales

Columns: transaction\_id, transaction\_date, transaction\_time, transaction\_qty, store\_id, store\_location, product\_id, unit\_price, product\_category, product\_type, product\_detail

Revenue = transaction\_qty \* unit\_price (you will need to calculate this)

Instructions

1. Read each question carefully before writing or running any SQL.
2. Write your SQL in Databricks then copy it into the code box provided, you can use microsoft word or google document to fill out this document.
3. Answer the written questions in the yellow boxes - these do NOT require SQL.
4. If a question says [WRITTEN], no Databricks needed.
5. There are 20 marks in total. Each question shows its mark value.
6.
  - Save a completed document as a pdf.
  - upload the pdf document to GitHub
  - Send your GitHub profile link, name, course name to ([mvuko@brightlearn.co.za](mailto:mvuko@brightlearn.co.za))

## Quick Reminder

% matches any sequence of characters (including none)  
\_ matches exactly one character  
[a-z] matches any single character in a range (Databricks uses RLIKE for full regex)  
LIKE is case-insensitive in most databases; NOT LIKE excludes matching rows

### Question 1 [1 mark]

Find all products where the `product_detail` starts with the word 'Dark'.

Write a SELECT query using `LIKE` that returns `transaction_id`, `product_detail`, and `unit_price`.

```
SELECT transaction_id, product_detail, unit_price
FROM coffee_sales
WHERE product_detail LIKE '%Dark'
LIMIT 10;
```

*How many distinct `product_detail` values start with 'Dark'? (run `COUNT DISTINCT` to find out)*

10

### Question 2 [1 mark]

Find all transactions where `product_detail` ends with 'Lg' (large size).

Return `transaction_id`, `product_detail`, `product_category`, and `unit_price`. Order by `unit_price` descending.

```
SELECT transaction_id, product_detail, product_category, unit_price
FROM coffee_sales
WHERE product_detail '%Lg'
ORDER BY unit_price DESC;
```

### Question 3 [1 mark]

Find all products that contain the word 'Chai' anywhere in `product_type`.

Return `product_type` and `product_detail`. Use `DISTINCT` so each combination appears once.

```
SELECT DISTINCT product_type, product_detail
FROM coffee_sales
WHERE product_type like %Chai%
ORDER BY product_detail;
```

### Question 4 [1 mark]

Use `NOT LIKE` to find all products that are NOT a type of tea.

Filter `product_category` so that it does NOT contain the word 'Tea'. Return `DISTINCT product_category`, `product_type`, and `product_detail`.

```
SELECT DISTINCT product_category, product_type, product_detail
FROM coffee_sales
WHERE product_category Not like %Tea%
ORDER BY product_category, product_type;
```

### Question 5 [1 mark]

Use the underscore wildcard to find products with a 2-character size code at the end.

The size codes are 'Sm', 'Rg', and 'Lg'. These all follow a space and are exactly 2 characters. Match `product_detail` values that end with ' \_\_' (space + any 2 chars). Return `DISTINCT product_detail` ordered alphabetically.

```
SELECT DISTINCT product_detail
FROM coffee_sales
WHERE product_detail like ' __'
ORDER BY product_detail ASC;
```

*[WRITTEN] What is the difference between % and \_ in a LIKE pattern? Give one example of each using this dataset.*

The \_ can be used to show how many number of characters are needed in the request made while % can mean as many characters as possible.

Main difference though is the number of characters they represent in a pattern.

E.g Where `product_detail` like ' \_\_'

Where `product_detail` like '%Sm'

### Question 6 [1 mark]

Use RLIKE (regex) to find all products where `product_detail` contains a number.

Return `DISTINCT product_detail`. (Hint: the regex pattern for 'contains a digit' is `[0-9]`.)

```
SELECT DISTINCT product_detail
FROM coffee_sales
WHERE product_detail LIKE '[0-9]%'
ORDER BY product_detail;
```

### Question 7 [2 marks]

Combine two wildcard conditions in one query.

Find transactions where `product_category` is 'Coffee' AND `product_detail` contains either 'Brazilian' or 'Ethiopian' (tip: use two OR conditions with LIKE).

Return `transaction_id`, `transaction_date`, `store_location`, `product_detail`, `unit_price`. Order by `transaction_date`.

```
SELECT transaction_id, transaction_date, store_location,
       product_detail, unit_price
FROM coffee_sales
WHERE product_category LIKE %Coffee%
      AND (product_detail LIKE %Brazilian%
           OR %Ethiopian%)
ORDER BY transaction_date;
```

### Question 8 [2 marks] [WRITTEN - no SQL needed]

*Explain in your own words what the following pattern matches: `product_detail LIKE '%Scone'` Then write a different LIKE pattern that would match product names that contain the word 'Blend' anywhere in the middle of the name.*

The first one returns product detail that ends with Scone,

Where `product_detail` like %Blend%



**B**

## Window Functions

Aggregate, Ranking, and Value functions | Questions 9 - 18

### Quick Reminder

Syntax: `function_name( column ) OVER ( PARTITION BY col ORDER BY col )`

Aggregate: SUM, AVG, COUNT, MIN, MAX - calculate across the window without collapsing rows

Ranking: ROW\_NUMBER, RANK, DENSE\_RANK, NTILE(n) - assign position within a partition

Value: LAG(col, n), LEAD(col, n), FIRST\_VALUE(col), LAST\_VALUE(col) - access other rows

### Part B1: Aggregate Window Functions

#### Question 9 [1 mark]

Calculate total revenue per store, keeping all individual rows.

Revenue = `transaction_qty * unit_price`. Use `SUM() OVER(PARTITION BY store_location)` to add a column called `store_total_revenue` to each row. Return `transaction_id`, `store_location`, `transaction_qty`, `unit_price`, and the new column. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    transaction_qty,
    unit_price,
    Sum(transaction_qty*unit_price) over (Partition by Store_location)AS store_total_revenue
FROM coffee_sales
LIMIT 20;
```

Looking at your results: does `store_total_revenue` change between rows in the same store? Why or why not?

Yes due to the fact that each quantity is not sold at the same price

#### Question 10 [1 mark]

For each transaction, show what percentage of its store's total revenue that transaction represents.

Calculate: `ROUND( (transaction_qty * unit_price) * 100.0 / SUM(transaction_qty * unit_price) OVER(PARTITION BY store_location), 2 )` as `pct_of_store_revenue`. Return `transaction_id`, `store_location`, `revenue` (calculated), and the percentage column. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2)*100.0 AS revenue,
    ROUND(SUM(transaction_qty*unit_price) over (partition by store_location), 2) AS
pct_of_store_revenue
FROM coffee_sales
LIMIT 20;
```

#### Question 11 [1 mark]

Calculate a running total of revenue per store, ordered by transaction date and time.

Add `ORDER BY transaction_date, transaction_time` inside the `OVER` clause of a `SUM()` window. This gives a cumulative (running) total that increases with each transaction. Return `transaction_id, store_location, transaction_date, transaction_time, revenue, running_total`. `LIMIT 30`.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    transaction_time,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND( SUM(transaction_qty * unit_price) OVER(partition by store_location
        order by transaction_date, transaction_time
            ), 2) AS running_total
FROM    coffee_sales
LIMIT  30;
```

*[WRITTEN] What is the key difference between the `SUM()` in Q9 (no `ORDER BY`) and the `SUM()` in Q11 (with `ORDER BY`)? What does adding `ORDER BY` change about how the window is calculated?*

Without an order by it just calculate the total revenue randomly while when calculating with the order by it adds order per transaction taking place on the data

## Part B2: Ranking Window Functions

### Question 12 [1 mark]

Rank every transaction within its store by revenue (highest first).

Use `ROW_NUMBER()` with `PARTITION BY store_location` and `ORDER BY revenue DESC`. Return `transaction_id, store_location, revenue, and row_num`. `LIMIT 30`.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROW_NUMBER() OVER(partition by store_location
        order by Revenue DESC) AS row_num
FROM    coffee_sales
ORDER BY store_location, row_num
LIMIT  30;
```

### Question 13 [2 marks]

Compare `RANK`, `DENSE_RANK`, and `ROW_NUMBER` side by side on the same data.

Rank products by `unit_price` within each `product_category`, highest price first. Include all three functions as separate columns. `LIMIT 40`. Look for ties (same `unit_price` in same category) and observe how each function handles them.

```
SELECT
    product_category,
    product_detail,
    unit_price,
    ROW_NUMBER() OVER( PARTITION BY product_category ORDER BY unit_price ) AS rn,
    RANK() OVER( PARTITION BY product_category ORDER BY unit_price) AS rnk,
    DENSE_RANK() OVER( PARTITION BY product_category ORDER BY unit_price) AS d_rnk
FROM    coffee_sales
```

```
ORDER BY product_category, unit_price DESC
LIMIT 40;
```

*[WRITTEN] Find a tie in your results (two rows with the same unit\_price in the same category). Write down the values of rn, rnk, and d\_rnk for those rows. Explain in one sentence why RANK and DENSE\_RANK give the same result for tied rows but differ for the row after the tie.*

It is because rank recognize one of the ties as a replacement of the number that was suppose to follow in chronology if the tie was not there but dense renk on the other had doesn't continues with the chronological order even after the tie

### Question 14 [1 mark]

**Use NTILE to divide transactions into 4 revenue quartiles within each store.**

Calculate `revenue = transaction_qty * unit_price`. Then use `NTILE(4) OVER(PARTITION BY store_location ORDER BY revenue ASC)` to assign a quartile (1 = lowest, 4 = highest). Return `transaction_id, store_location, revenue, quartile`. LIMIT 30.

```
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    Ntile(4) over (partition by store_location order by revenue ASC )AS quartile
FROM    coffee_sales
ORDER BY store_location, revenue ASC
LIMIT 30;
```

## Part B3: Value Window Functions

### Question 15 [1 mark]

**Use LAG to compare each transaction's revenue to the previous transaction in the same store.**

Order by `transaction_date, transaction_time`. Use `LAG(revenue, 1, 0)` - the third argument 0 is the default value when there is no previous row. Return `transaction_id, store_location, transaction_date, revenue, prev_revenue`. LIMIT 20.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND( LAG(Revenue,1,0) OVER( Order by transaction_date, transaction_time

    ), 2) AS prev_revenue
FROM    coffee_sales
LIMIT 20;
```

### Question 16 [1 mark]

**Use LEAD to show the next transaction's revenue alongside the current one.**

Same partition and order as Q15. Use `LEAD(revenue, 1)`. Return `transaction_id, store_location, transaction_date, revenue, next_revenue`. LIMIT 20.

```

SELECT
    transaction_id,
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND( LEAD(revenue,1) OVER(order by transaction_date,transaction_time
    ), 2) AS next_revenue
FROM    coffee_sales
LIMIT   20;

```

*[WRITTEN] What value does next\_revenue show for the very last transaction of each store partition? Why?*

It show that the company is making more profit

### Question 17 [1 mark]

Use **FIRST\_VALUE** to show the cheapest product in each category alongside every row.

Use **FIRST\_VALUE**(product\_detail) OVER(PARTITION BY product\_category ORDER BY unit\_price ASC). Return product\_category, product\_detail, unit\_price, and cheapest\_in\_category. Use **DISTINCT** to avoid too many duplicate rows. **LIMIT 30**.

```

SELECT DISTINCT
    product_category,
    product_detail,
    unit_price,
    FIRST_VALUE(product_detail) OVER( partition by product_category
                                   order by unit_price ASC
    ) AS cheapest_in_category
FROM    coffee_sales
ORDER BY product_category, unit_price
LIMIT   30;

```

### Question 18 [3 marks] - Challenge

This question combines wildcards AND window functions. Take your time - re-read the question before writing SQL.

**Part A (1 mark):** Filter the data to only include rows where product\_detail ends in 'Lg' or 'Rg' (large or regular sizes). Use **LIKE** with an **OR** condition. Assign revenue as transaction\_qty \* unit\_price.

**Part B (1 mark):** On that filtered data, use **RANK()** OVER(PARTITION BY store\_location ORDER BY revenue DESC) to rank each transaction within its store.

**Part C (1 mark):** Also add the store's average revenue using **AVG()** OVER(PARTITION BY store\_location) as store\_avg\_revenue.

Return: transaction\_id, store\_location, product\_detail, revenue, store\_rank, store\_avg\_revenue. **LIMIT 40**.

```

SELECT
    transaction_id,
    store_location,
    product_detail,
    ROUND(transaction_qty * unit_price, 2) AS revenue,

```



```
Rank() over (Partition by store_location order by transaction_qty*unit_price DESC AS
store_rank,
ROUND(AVG(transaction_qty*unit_price) OVER (partition by store_location), 2) AS
store_avg_revenue
FROM coffee_sales
WHERE product_detail like '%Lg'
OR product_detail like '%Rg'
ORDER BY store_location, store_rank
LIMIT 40;
```

*[WRITTEN] Look at your results. Does every store\_avg\_revenue change between rows? Explain why or why not. Then explain in one sentence what store\_rank = 1 means for each store.*

Filtered to Lg/Rg product only Ranked by Revenue within store Top Revenue:\$9.50 and average per store is for the Lg/Rg products is \$4.68

**End of Exercise - Well done!**

Total marks: 20